



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ПРОГРАММИРОВАНИЕ В ЗАДАЧАХ ХТ

Лекция 15. ООП в Python

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

-  Объектное ориентированное программирование
-  Принципы ООП
-  Особенности ООП в Python
-  Создание классов
-  Наследование
-  Полиморфизм

Объектно-ориентированное программирование

- ☎ Метод программирования, основанный на:
 - ☉ понятии объекта, как формально описанной совокупности данных и процедур;
 - ☉ реализации алгоритмов, как описания взаимодействия объектов между собой
 - ☉ понятии класса объектов и описании иерархий классов*
*(кроме ООП языков, основанных на прототипах)
- ☎ Лучшие практики разработки программного обеспечения
 - ☉ Использование ООП позволяет создать расширяемое, масштабируемое и легкоуправляемое ПО
- ☎ Тесная связь модели компьютерной системы с объектами реального мира

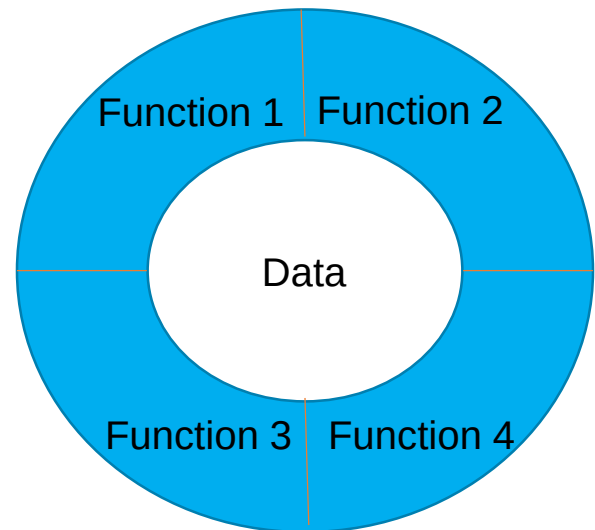
Понятие объекта

- Представим сущность из реального мира, например, компьютер
- С компьютером связаны:
 - Набор данных, таких как объем памяти, количество процессорных ядер, параметры операционной системы и т.д.
 - Набор действий, которые могут быть выполнены с компьютером, такие как включение, выключение, добавление модулей памяти и т.д.
- Сущность реального мира может быть смоделирована в рамках языка программирования, такая модель называется объектом



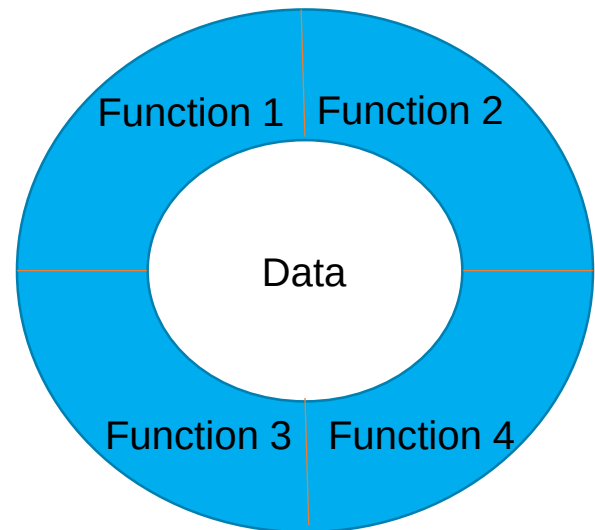
Что такое объект? (1/2)

- 🌀 Объект – модель сущности, которая состоит из данных и управляющих ими функций
 - 🌀 Объекты – это материальные сущности, бизнес сущности или абстракции
 - 🌀 Свойства или состояние сущности представлены данными объекта
 - 🌀 Функции объекта моделируют действия, которые могут быть выполнены сущностью /с сущностью
 - 🌀 Совокупность функций объекта называют его поведением



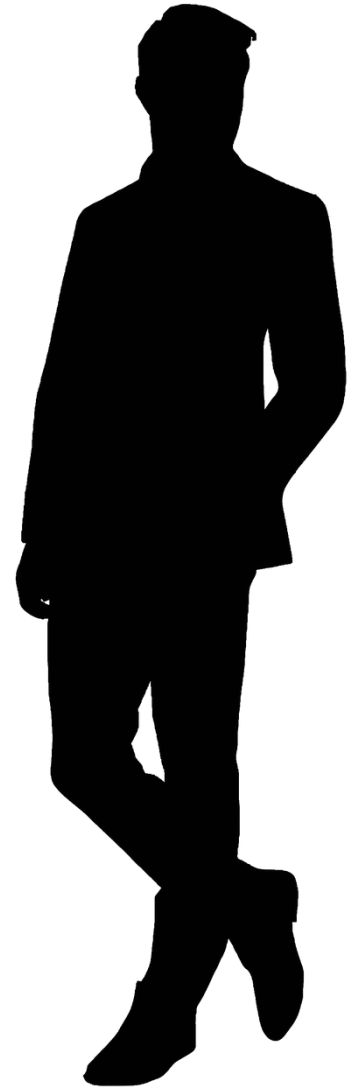
Что такое объект? (2/2)

- ☉ Данные и реализации функций инкапсулируются в объекте
 - ☉ Инкапсуляция означает замкнутость или скрытость
 - ☉ Доступ к данным для чтения или изменения осуществляется только посредством функций этого объекта
 - ☉ Детали реализации функций скрыты от вызывающей стороны



Пример объекта

- 📡 Сущность - человек
- 📡 Свойства: Имя, Фамилия, Возраст...
- 📡 Поведение: Ходит, Говорит, Работает...
- 📡 Как узнать, сколько у человека при себе денег?



Процедурный подход

getBalance



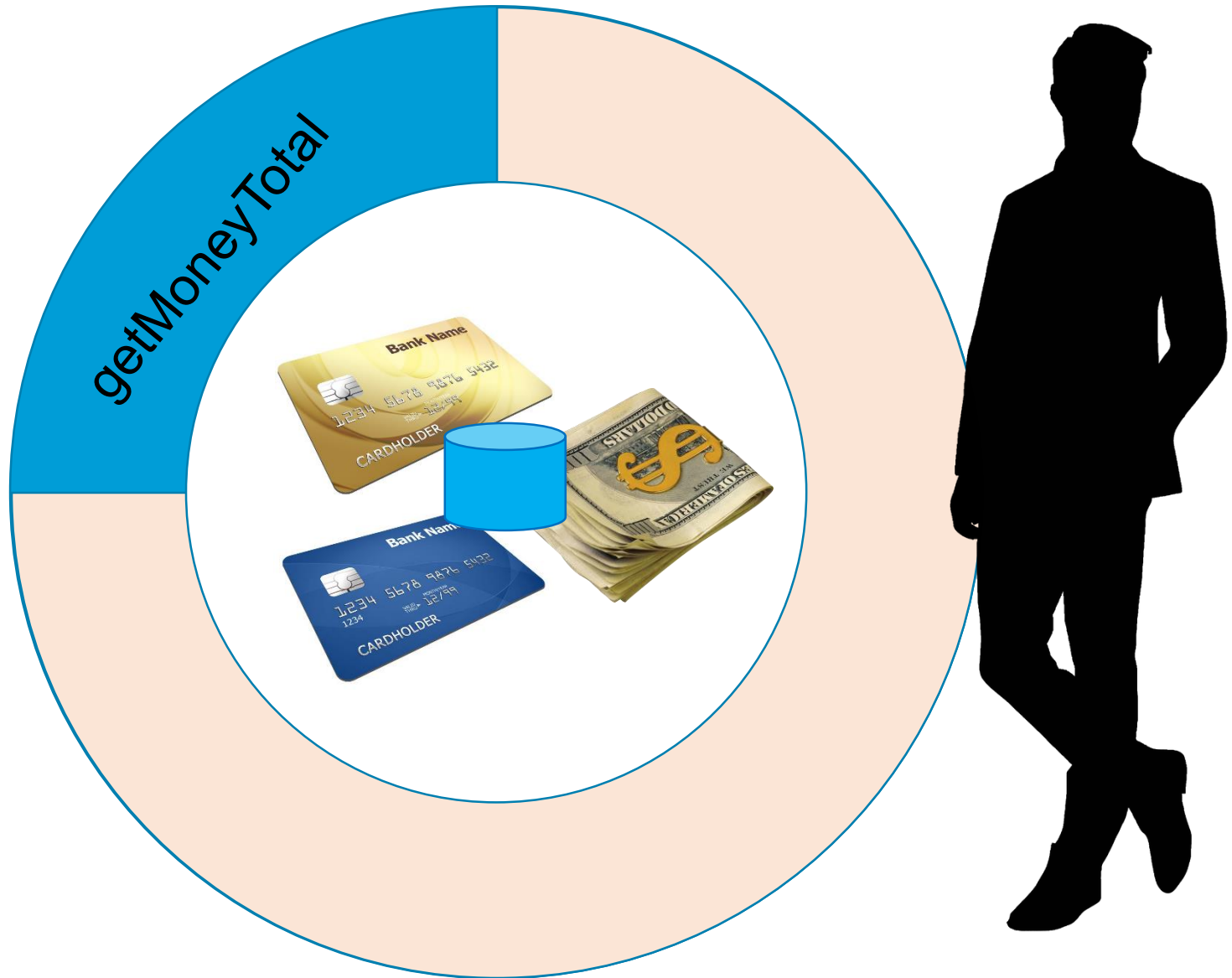
getAccountInfo



getCash

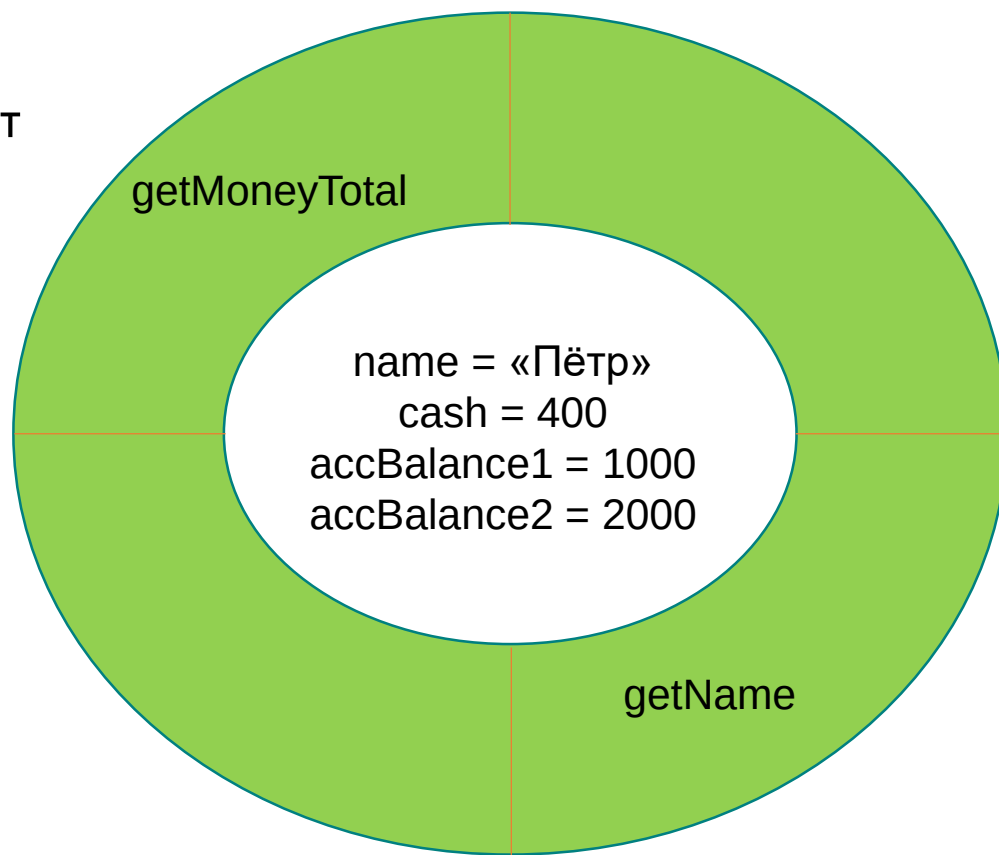


Объектно-ориентированный подход



Объект: Человек

- Данные инкапсулированы внутри объекта, прямой доступ к ним извне объекта закрыт
 - Если мы хотим узнать значение поля name, мы должны использовать функцию getName
- Функции объекта позволяют получить его состояние или изменить его
- При проектировании приложения мы моделируем значимое для нас поведение объекта в виде бизнес-функций
 - getMoneyTotal



Сообщения и методы (1/2)

Сообщение

- ⦿ Сообщение - это запрос на выполнение объектом действия (или предоставление им услуги)
- ⦿ Обязанность объекта - выбрать процедуру, соответствующую сообщению, и выполнить ее код

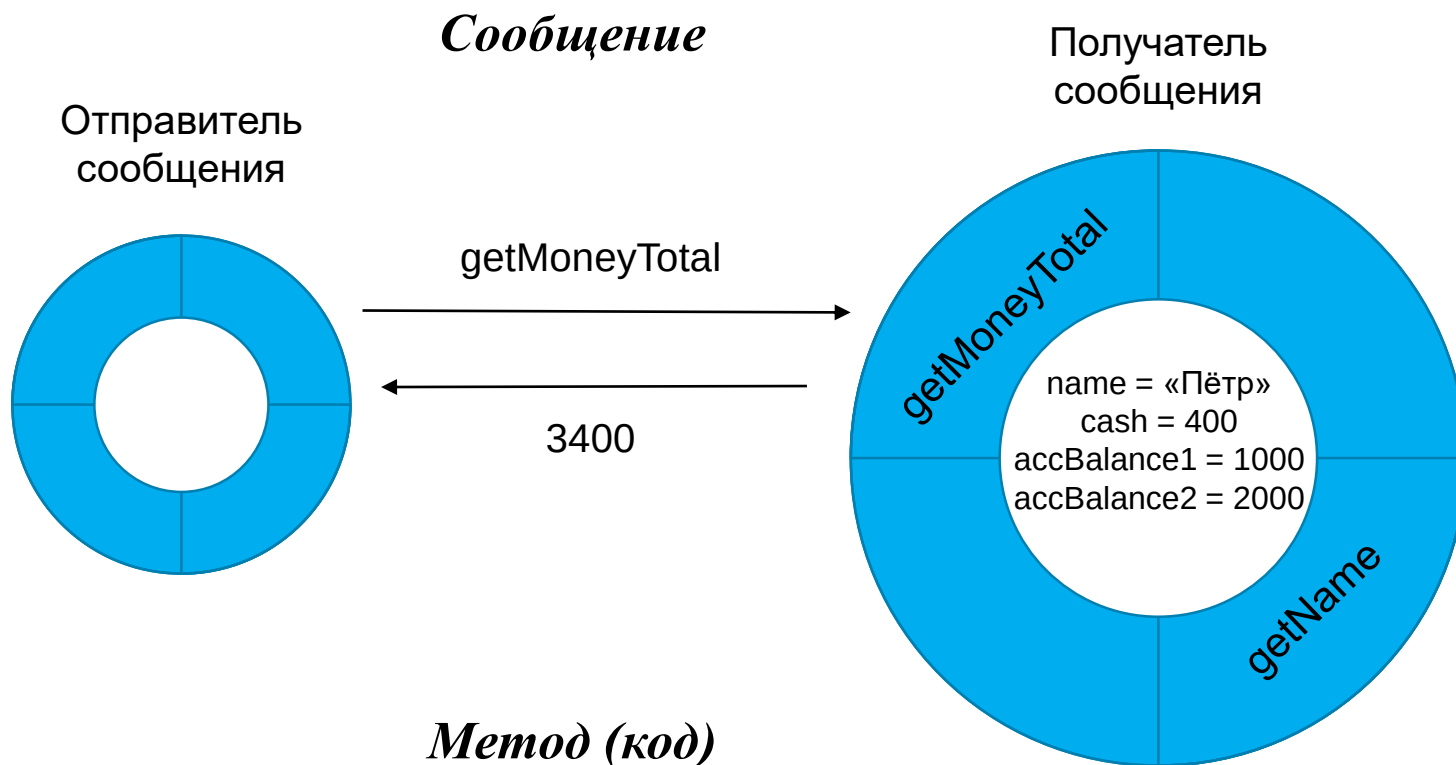
Метод

- ⦿ Метод - это реализация действия, запрошенного в сообщении
- ⦿ Метод – процедура, являющаяся частью поведения объекта, содержащая код, реализующий алгоритм действия

Динамический вызов / обработка сообщения

- ⦿ Выбор метода, который будет вызван, определяется во время выполнения

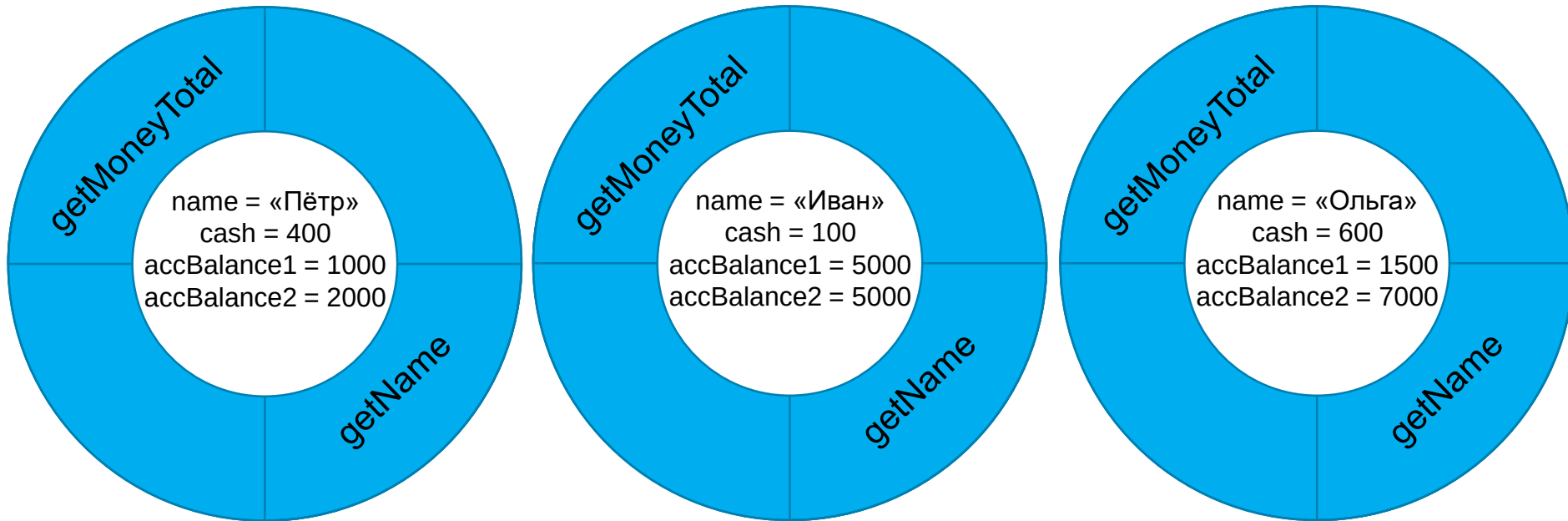
Сообщения и методы (2/2)



Доступ к данным

- ☎ Доступ к данным, хранящимся в объектах, другие объекты могут получить непосредственно* или посредством отправки сообщений
 - ☉ Прямой доступ нарушает принцип инкапсуляции
- ☎ Рекомендуемый способ – отправка сообщений
 - ☉ Сообщения позволяют разработчикам скрыть реализацию методов и детали внутреннего представления данных
- ☎ Обычно объект предоставляет набор методов, позволяющих получить доступ к его данным
 - ☉ Например, используя сообщение `getMoneyTotal`, мы узнаем количество денег у человека, при этом не зная, как рассчитывается сумма и из каких источников берутся данные для расчета

Объекты и классы



- Три объекта имеют одинаковую структуру данных и поведение
- Значения данных свои для каждого человека
- Переменные, необходимые для хранения данных, одинаковы во всех случаях; объекты отвечают на одни и те же сообщения
 - Все три объекта – экземпляры одного и того же класса объектов

Классы и экземпляры (1/2)

- Класс – множество объектов, представляющих однотипные сущности
 - Яблоко, апельсин, груша – это фрукты
- Экземпляр класса – объект представитель класса
 - Яблоко – представитель класса фрукты
 - Экземпляр и объект в ООП – синонимы



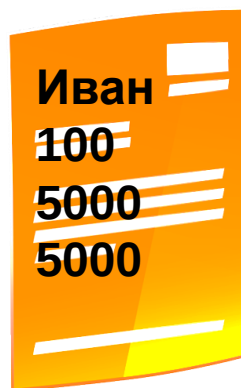
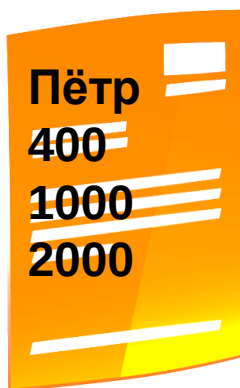
Классы и экземпляры (2/2)

📡 Класс определяет структуру, единую для всех экземпляров этого класса:

- ⦿ Набор полей данных
- ⦿ Набор методов
 - Сообщения, которые может получать экземпляр
 - Реализация обработки сообщений



📡 Экземпляры класса идентичны по структуре, но могут отличаться значениями полей данных



Основные свойства ООП

Модульность

- ⦿ Код разбит на модули: прототипы или классы

Инкапсуляция

- ⦿ Можно скрыть ненужные внутренние подробности работы объекта от окружающего мира

Наследование

- ⦿ Можно создавать специализированные классы на основе базовых

Полиморфизм

- ⦿ В разных объектах одна и та же операция может выполнять различные функции

Композиция*

- ⦿ Объект может быть составным и включать в себя другие объекты

ООП в Python

- 📡 Все сущности - объекты
- 📡 Классы – пользовательские типы
- 📡 Инкапсуляция 😞
 - 🕒 Все данные открыты
 - 🕒 Исключение: __
- 📡 Наследование 😊
 - 🕒 Множественное наследование
 - 🕒 Duck typing
- 📡 Полиморфизм 😊
 - 🕒 Все методы виртуальные
 - 🕒 Динамическая типизация
 - 🕒 Duck typing

Создание классов

- 📡 Классы объявляются с использованием ключевого слова **class**
- 📡 Переменные и функции, объявленные в теле класса - члены класса
 - 🕒 Переменные называют атрибутами или полями данных
 - 🕒 Функции называют методами

```
class My_class: # Минимальное объявление класса
    pass
```

```
class Person:
    name = "John Doe"
    age = 20
    def birthday(self): # self – обязательный первый параметр
        self.age += 1 # self – ссылка на экземпляр класса
```

```
p = Person() # Создание экземпляра класса - объекта
```

Изменение классов

- 📡 Классы в Python не являются неизменными
- 📡 После их объявления в них можно переопределить члены или добавить новые
- 📡 Методы класса обязательно получают в качестве первого параметра ссылку на экземпляр класса, которому они принадлежат
 - 🕒 При вызове метода этот параметр явно не задается

```
def sing(self, song):  
    print(song)  
  
Person.sing = sing  
  
p = Person()  
p.sing("La-la-la") # La-la-la
```

Инициализация экземпляра класса

- 🌀 Конструктор – метод с именем `__init__` неявно вызывается при создании экземпляра класса

```
class Person:
    def __init__(self, name, age)
        self.name = name
        self.age = age

    def birthday(self):
        self.age +=1

p1 = Person("John Doe", 20)
p2 = Person("Jane Doe", 18)

print(p1.name, p1.age) # John Doe, 20
print(p2.name, p2.age) # Jane Doe, 18
```

Инициализация по умолчанию

- 🌀 Параметры конструктора могут иметь значения по умолчанию

```
class Person:
    def __init__(self, name = "John Doe", age = 20)
        self.name = name
        self.age = age

    def birthday(self):
        self.age += 1
        age = 50 # Значение будет переопределено конструктором

p1 = Person()
p2 = Person("Jane Doe", 18)

print(p1.name, p1.age) # John Doe, 20
print(p2.name, p2.age) # Jane Doe, 18
```

Деструктор

- Метод `__del__` автоматически вызывается при удалении экземпляра класса (если объявлен)
 - Нужен только для классов, использующих внешние ресурсы

```
class Person:
    def __init__(self, name = "John Doe", age = 20):
        self.name = name
        self.age = age
    def __del__(self):
        print(self.name + " is dead")

    def birthday(self):
        self.age += 1

p = Person()
del p # John Doe is dead
```

Наследование

- При объявлении класса можно указать его класс-родитель
 - Классы-наследники содержат все члены классов-родителей
 - Классы-наследники могут переопределяют методы классов-родителей
 - Переопределенные методы доступны через `super()`

```
class Student(Person):  
    def __init__(self, name, age, year):  
        super().__init__(name, age)  
        self.year = year
```

```
s = Student("Jane Doe", 18, 2)
```

```
print(s.name, s.age, s.year)  
s.birthday()  
print(s.age)
```


Полиморфизм

- 📡 Полиморфизм (от греч. πολύ — «много», морφή — «форма») — принцип ООП, позволяющий использовать один набор методов (интерфейс) для работы с разными типами данных
 - 📡 Объекты разных классов могут обрабатываться единообразно через общий интерфейс
 - 📡 Один и тот же метод (или оператор) может вести себя по-разному в зависимости от типа объекта

📡 Виды полиморфизма

- 📡 Перегрузка функций/операторов
 - Один и тот же метод/функция работает по-разному в зависимости от типов аргументов: $1 + 2 = 3$ в тоже время $"a" + "b" = "ab"$
- 📡 Параметрический полиморфизм (универсальные шаблоны)
 - Функции/классы могут работать с разными типами данных без их явного указания
- 📡 Полиморфизм подтипов (наследование и переопределение методов)
 - Дочерние классы переопределяют методы родительского класса.

Полиморфизм через наследование

```
class Animal:
    def speak(self):
        raise NotImplementedError(
            "Метод должен быть переопределён!")

class Dog(Animal):
    def speak(self):
        return "Гав!"

class Cat(Animal):
    def speak(self):
        return "Мяу!"

animals = [Dog(), Cat()]

for animal in animals:
    print(animal.speak()) # Вывод: Гав! Мяу!
```

Полиморфизм через перегрузку операторов

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

v1 = Vector(1, 2)
v2 = Vector(3, 4)
v3 = v1 + v2  # Полиморфизм: оператор + работает по-новому
print(v3.x, v3.y)  # Вывод: 4 6
```

Полиморфизм через утиную типизацию

```
class Car:
    def move(self):
        return "Двигается на колёсах"

class Boat:
    def move(self):
        return "Плывёт по воде"

def travel(vehicle):
    print(vehicle.move())

car = Car()
boat = Boat()

travel(car)    # Вывод: Двигается на колёсах
travel(boat)   # Вывод: Плывёт по воде
```

Итоги

В данной лекции были рассмотрены следующие темы:

-  Объектно ориентированное программирование
-  Принципы ООП
-  Особенности ООП в Python
-  Создание классов
-  Наследование
-  Полиморфизм